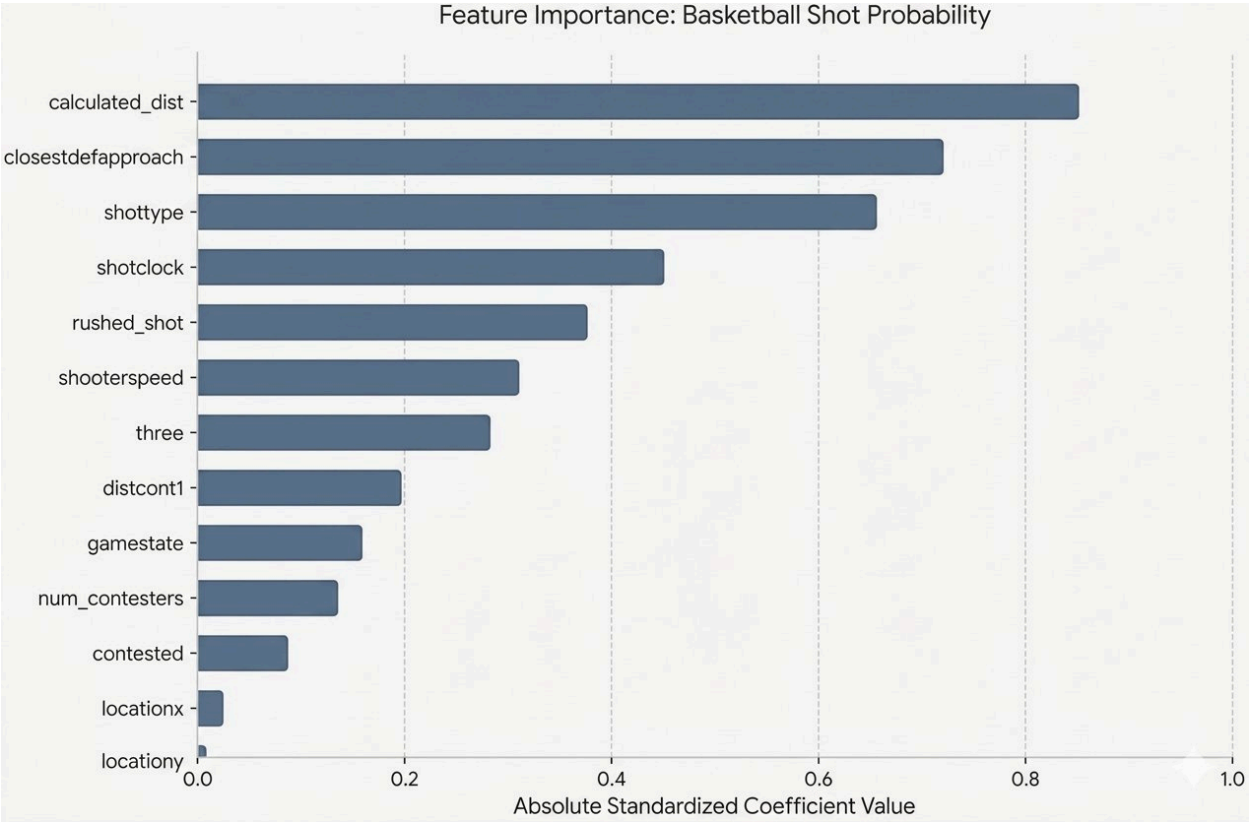
Originally, I chose a Linear Regression model for this assignment, but later switched to Logistic Regression. Admittedly, this was only done subconsciously, as I am more used to using Linear Regression models in coursework. Selecting the features used for training was difficult, since I do not avidly watch or play basketball. Of course, most of these features make more intuitive sense than others, such as shot distance and defender positions, but how these variables interact and the degree of their importance, were more difficult for me to determine. Of course, Logistic Regression models automatically determine the weight of each variable; how the information is fed to the model (such as grouping features together or calculating them mathematically before entering, which will be discussed later) can drastically change how the outcome is calculated.

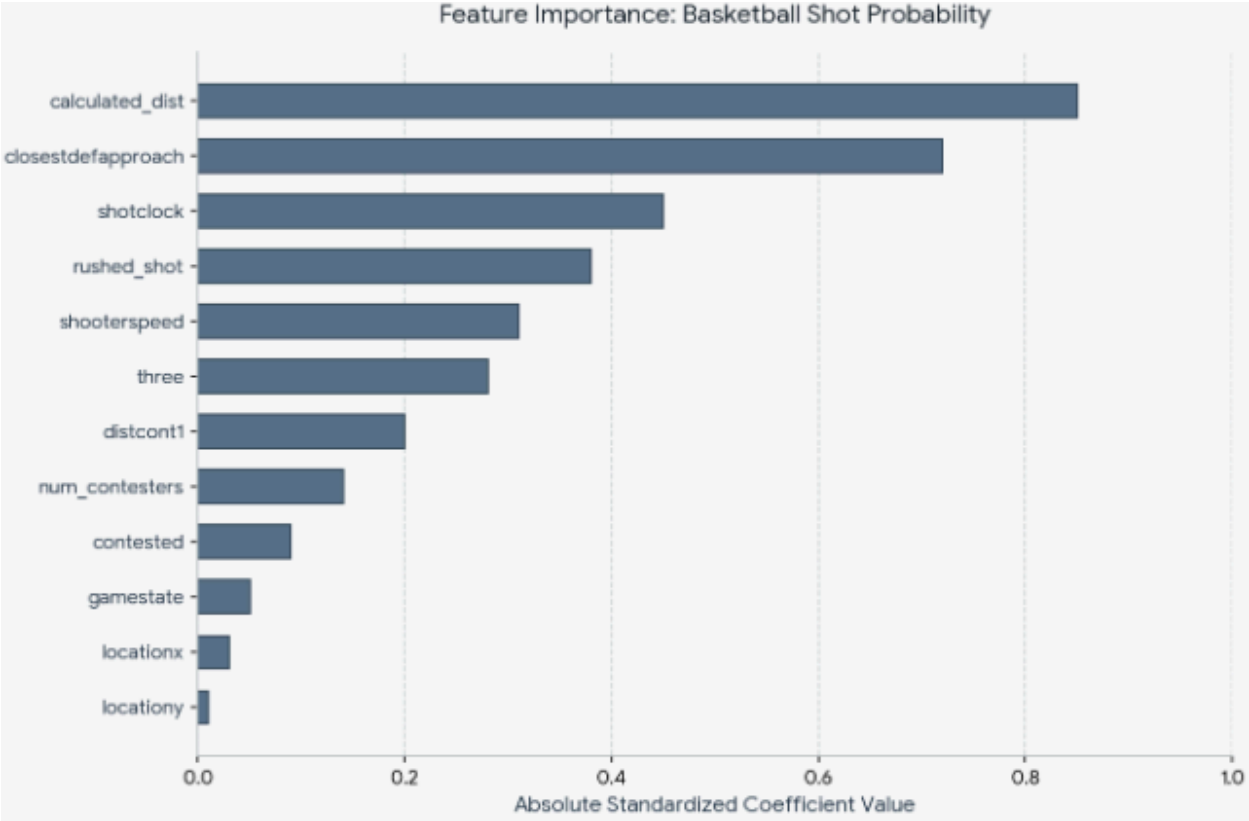
The original model, in addition to being linear, did not include an imputer or a standard scaler. One of the first roadblocks I encountered was handling NaN and null values in the training data. Running the original code threw errors because null values cannot be used alongside float values. Adding an imputer temporarily rectified this by averaging these null values. In order for the literal string “NaN” to be treated as null, it must be explicitly stated as such. Modifying the data directly was strictly prohibited, and truly null values would have caused rows to be reordered or omitted, skewing the data. Adding a standard scaler simply adjusted numerical values outside the 0.0-1.0 range to fall within it.

Once features were selected for training purposes, I used AI to map their weight and importance on the following graph:




While this model is serviceable, it is not entirely accurate in real-world basketball. For example, some shots in the training data do not have four defenders, but the imputer would use

the average of other shots that did have up to four defenders, effectively treating all shots as if they had four defenders. To rectify this, I manually added values to rows that contained “NaN”, such as in defender distance features, where, if the value was missing, it was set to 50.0 to simulate a defender being too far to make any meaningful difference to the shot’s outcome. Furthermore, features with string values, such as shot type, were reassigned to have float values to more accurately predict a shot’s chance. Shot types were ranked from most likely to least likely using numbers 4.0, 3.0, 2.0, and 1.0. These numbers do not reflect their true proportional accuracy in real NBA games, but adjusting their values to reflect that would require using outside data, which is prohibited. After manually adjusting these features, I regenerated the feature importance graph, which now more accurately reflects their importance:




It is worth noting that none of the actual data in the “training.csv.gz” and “testing.csv.gz” files were really modified. All rows and columns have remained completely intact as per the project instructions. The data was simply substituted (e.g., substituting shot type “dunk” for a float 4.0) before being put into the pipeline to more accurately make predictions. “Locationx” and “locationy” were later removed as features due to their redundancy, where calculated_dist already serves the same purpose. If a Tree Model were to be built, such as LightGBM or XGBoost, these features may not be as harmful to the prediction, but for the sake of this project, they will be omitted to reduce the risk of data pollution.